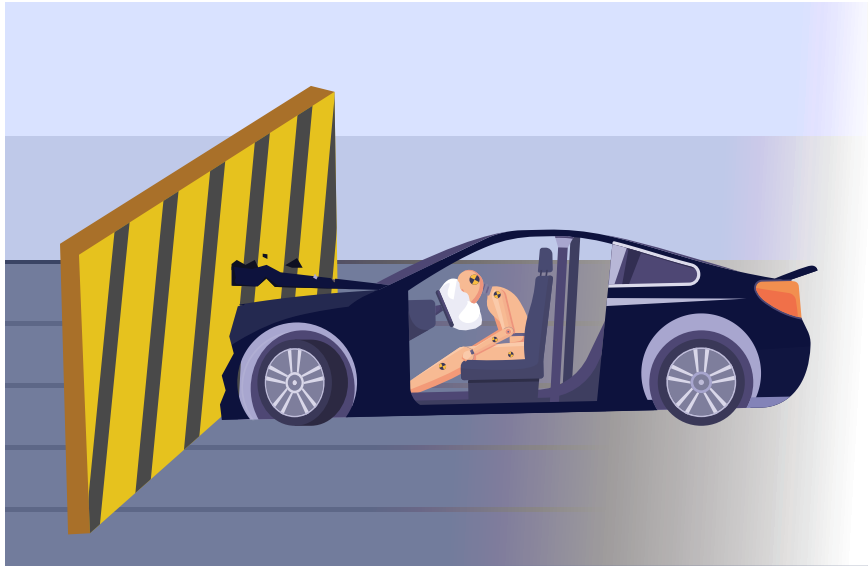
Simulation

07

Simulations play a key role in technology by helping us analyse and visualise complex systems. They are used to model processes such as traffic patterns or environmental changes, offering insights into how various factors interact and affect outcomes.

Simulations is a powerful tool in modern science and technology, allowing us to model real-world systems and processes in a controlled virtual environment. By replicating natural or human-made phenomena digitally, simulations provide valuable insights into how these systems function and respond to various conditions, offering new ways to understand and interact with them.



As technology advances, the need for accurate and efficient simulations continues to grow across diverse fields, including education, research, and industry. For example, when engineers need to test the safety of a new car design, they use simulations to model various crash scenarios instead of physically building and crashing multiple prototypes. This approach lowers costs, saves time, and eliminates risks while identifying potential design flaws. As a result, simulations boost both efficiency and innovation while improving safety in product development. From scientific experiments to video games, simulations open up a realm of possibilities, helping us learn, innovate, and solve real-world problems.

Simulation is a broad field, with its nature varying greatly depending on the application. One such application is the recreation of natural phenomena, such as fire, through digital animation. In this lesson, we will build upon the particle system created earlier and explore how coding can simulate complex behaviours like fire, demonstrating how procedural animation can bring these dynamic processes to life.

Activity

Now, let us create a simulation of fire using p5js.

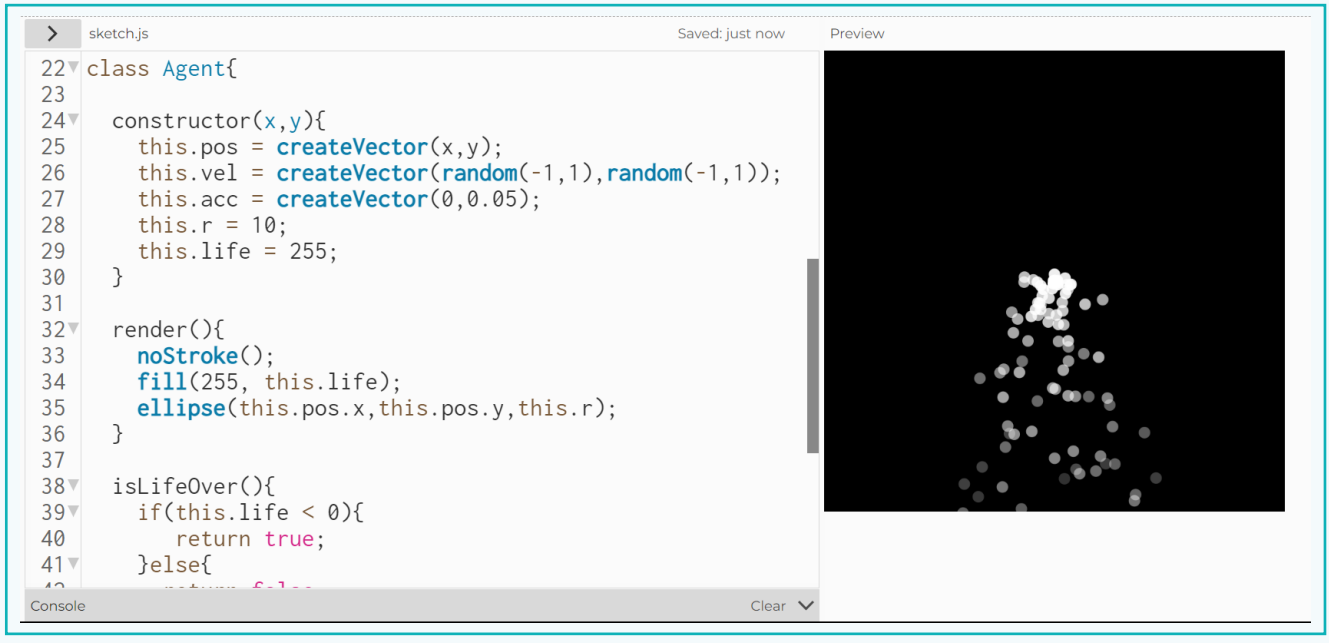
We will build this activity upon the foundation established in the previous lesson, where we created an Agent class for simulating the movement of particles with physical properties. We also created a particle system using the objects created from the class. Now, we will take it further to create a fire effect, which is commonly used in simulations, games, and virtual platforms.

Activity

Part 1 - Open Code Editor

1. Open the p5js website https://p5js.org/ and click on 'Start Coding' to open an online editor.

2. Open the activity created in the lesson - 'Applications of Vectors' from the saved file.



In case the file is not saved accidentally, you can retrieve the code here. (Ensure to save the Word file on the server.)

Activity

Part 2 - Update Class Properties

With the existing particle system, our goal is to create a fire-like effect. Since fire flames typically rise upwards and are unaffected by gravity, we will need to adjust the initial velocity settings. Save a copy of the project and rename it “simulation”.

1. Update the values of the *y* component in the *velocity* vector *this.vel* within the class to (-5, -1).
   - Update the particle size—*this.r* to ‘40’.
   - To replicate the colours of fire, such as shades of red, orange, and yellow, update the *fill()* command so that the green and alpha components of the (R, G, B, A) values vary with the particle’s lifespan.

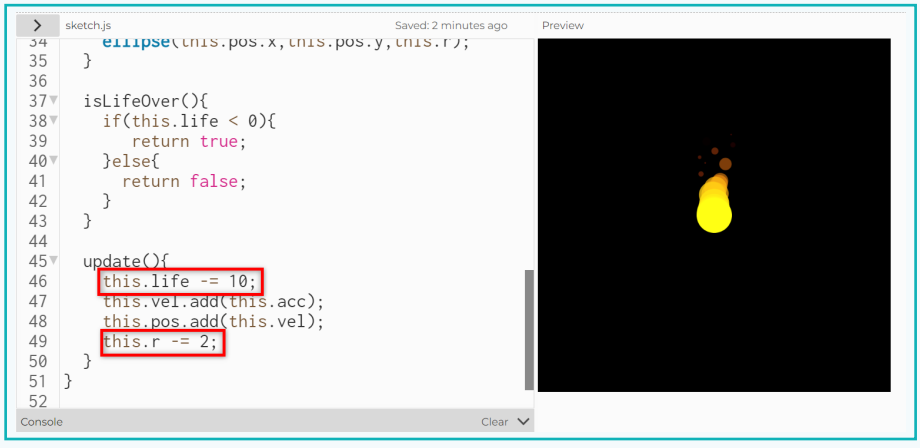
Part 3 - Update the Dynamics

As the particles move away from the centre, their size should gradually decrease to simulate realistic flame behaviour. In actual flames, the plasma shrinks as it rises from its source. Additionally, the particle’s lifespan should shorten at a faster rate to simulate the quick fading of flame particles.

Activity

1. Use the 'decrement' operator to reduce *this.r* by '2' in the *update()* method.

2. Change the decrement quantity of *this.life* variable by '10'.

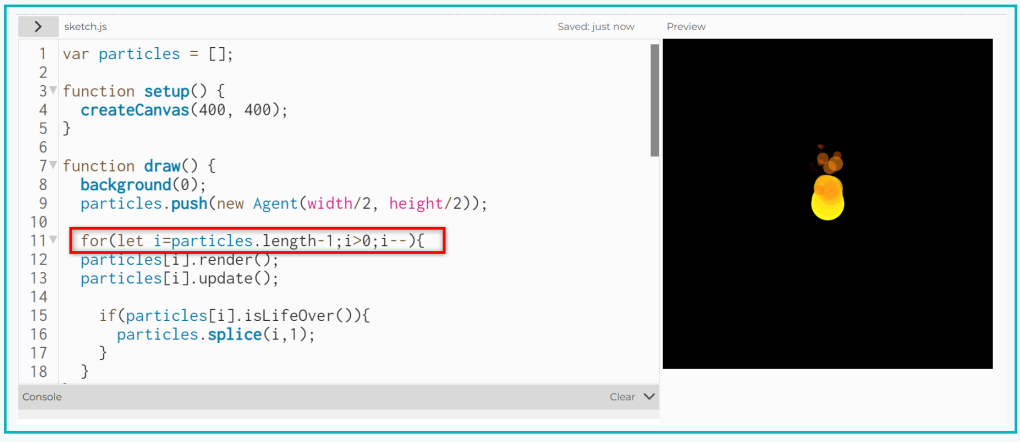


The output resembles fire, but it still requires some fine-tuning.

Remember that we created an array of particles and need to loop through all objects in the array to call the *render()* and *update()* methods. Currently, the loop starts with *i = 0* and increments up to the last particle. If we start the counter from the last element in the array and decremented downwards, new particles will not overlap previous ones.

3. Update the *for* loop to start from the last array element with *i= particles.length - 1.*

   - Include a condition so that the loop continues while *i > 0*.
   - Add a 'decrement' operator to 'i' to move backwards through the array.



Activity

Part 4 - Increase the Number of Particles

To make the fire visuals more detailed and realistic, we can increase the number of particle objects. Currently, a single particle is created each time the *draw()* function is called. We can create a loop to create multiple particle objects in every cycle to provide a more dynamic fire effect.

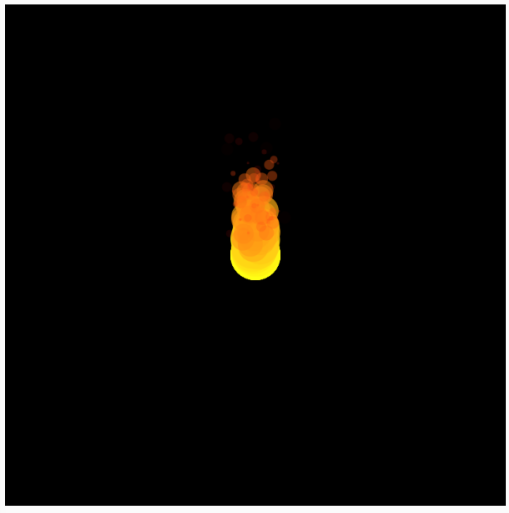
- Create a for loop around the code where we created the new object and push it into the array.
- We can create up to 5 objects using a loop.

sketch.js — Saved: just now

```
1  var particles = [];
2
3  function setup() {
4    createCanvas(400, 400);
5  }
6
7  function draw() {
8    background(0);
9
10   for(let i=0;i<5;i++){
11   particles.push(new Agent(width/2, height/2));
12   }
13
14   for(let i=particles.length-1;i>0;i--){
15   particles[i].render();
16   particles[i].update();
17
18     if(particles[i].isLifeOver()){
```

Console — Clear

Preview



We can observe that increasing the number of elements in the array makes the fire appear more continuous and flowing.

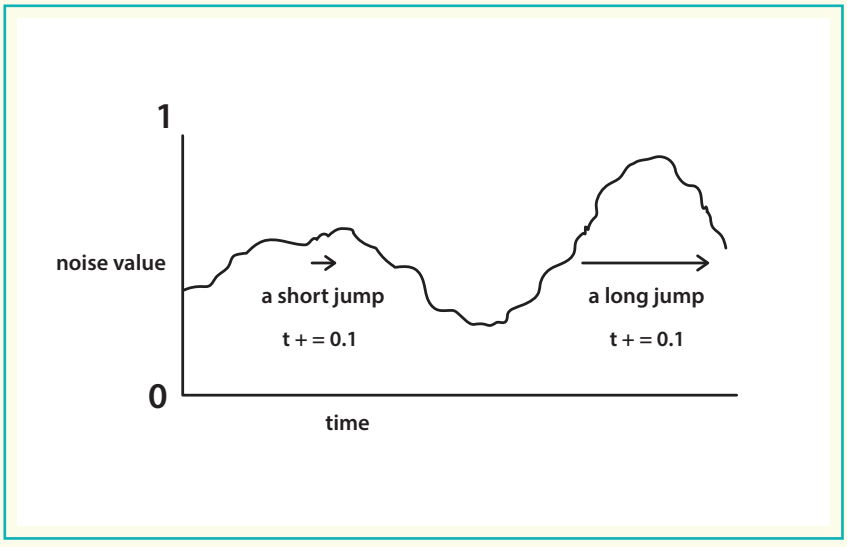
Part 5 - Perlin Noise

To enhance the current particle system to better simulate a flame, we can use the **noise()** function, which generates values based on Perlin Noise. Let us revisit some key concepts for better implementation.

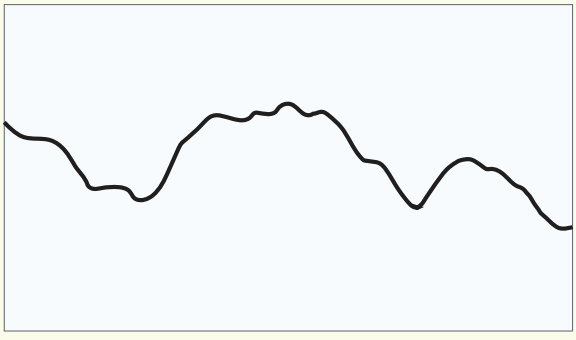
Activity

Recall:

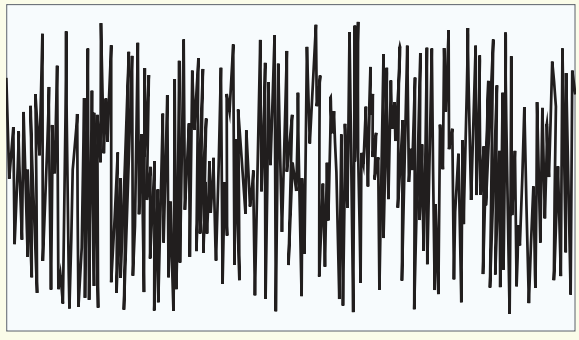
- **Perlin Noise:** Perlin noise is used to simulate various natural phenomena by generating naturally ordered random numbers.
- **noise():** The *noise()* command generates the Perlin noise, which is a smooth, continuous random noise that is commonly used to create natural-looking patterns. It produces a smooth, random value between 0 and 1 based on a single input.



As this input value changes gradually, the random output changes, creating a natural, continuous flow rather than abrupt, unpredictable jumps.



Perlin Noise



Random Noise

Activity

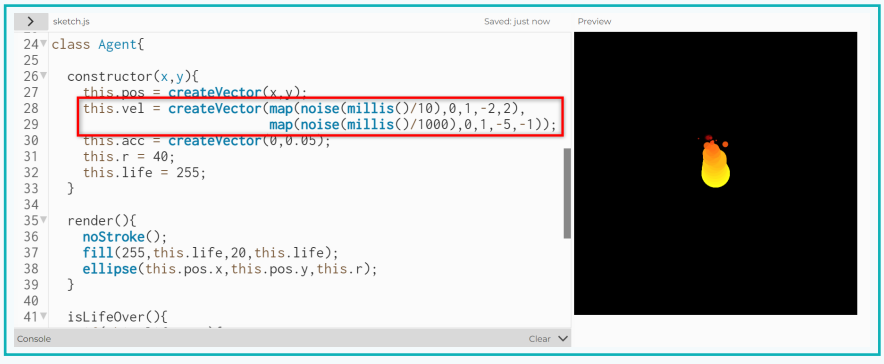
Unlike *random()*, which generates completely unpredictable values, *noise()* creates smooth transitions between values, making it ideal for animations and procedural textures.

map() - The *map()* command re-maps a number from one range to another, making it useful for scaling a value from an input range to fit within a different output range.

millis() - The *millis()* command returns a number representing the elapsed time in milliseconds. Since *millis()* continuously increases, it can be used as input to the *noise()* command.

Using these concepts, we can enhance the particle system to create smoother and more realistic flames. By integrating Perlin noise, the movement of the particles will more closely mimic the natural, fluid dynamics of real fire.

1. We will add the *noise()* command with input as fractions of the *millis()* command. Since *millis()* generates an incrementing number for every millisecond that passes, it is useful as an input to the *noise()* command
2. Use the *map()* command to map the output of the *noise(millis()/10)* command from the range (0,1) to (-2,2) for the x component of velocity.
3. Use the *map()* command to map the output of the *noise(millis()/1000)* command from the range (0,1) to (-5,-1) for the y component of velocity.



Output Video -

Activity

Part 6 - Varying Parameters

Adjusting parameters like the initial velocity, shape, and size of the particles, and input to the *noise()* and *map()* commands, can provide us with different versions of flames. Let us try simulating the candle flame which is calmer and has a shape like a cone.

1. Update the number of objects created per code execution cycle in the *for* loop from '5' to '10'. This will increase the number of particles, improving the continuity and smoothness of the flame.

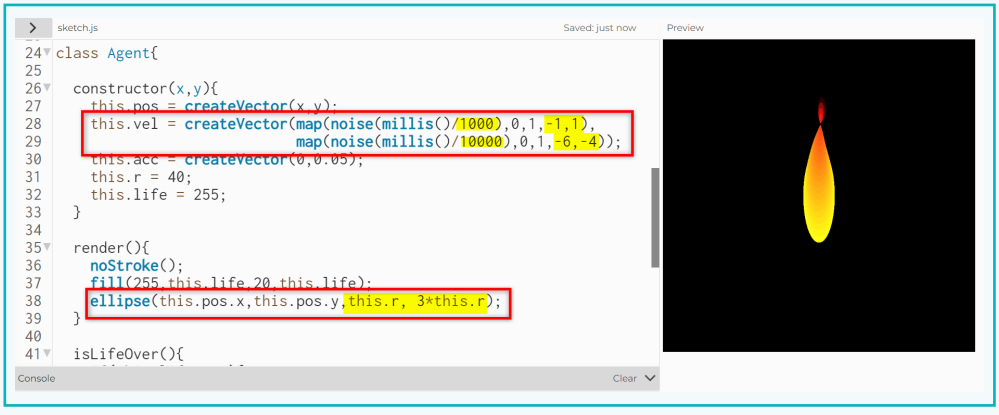
```
function draw() {
  background(0);

  for(let i=0;i<10;i++){
  particles.push(new Agent(width/2, height/2));
  }
```

2. Update the input of the *noise()* command using output values of *millis()* command by higher values.
   - For example, (millis()/1000) generates values— 0.001, 0.002, ... and so on every milli second.
   - This will reduce the step size of the change in input value to the *noise()* command, providing smoother Perlin noise values.

3. Change the range of initial velocity values as shown to limit the vibrant movement of the flame.

4. Change the shape of the particle from a circle to an ellipse for a flame-like shape.



Activity

Our simulation is almost complete. At the top of the flame, we notice a small, unintended flame. This happens because of the value of *this.r* decreases, it eventually becomes negative, causing a shape to be drawn with that value. We can rectify this using a simple condition that will limit the minimum value of the *this.r* variable to '0'.

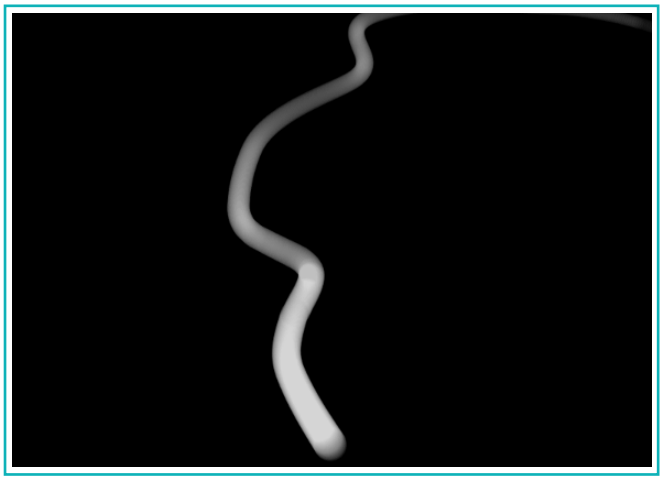
- Add a condition such that if *this.r* is less than 0, set its value to '0' in the *update()* method.

Output -

Output Video -

We can create different versions of fire by adjusting the parameters that have been updated, such as the rate of change in particle size and lifespan. Below are several versions of fire created by modifying these parameters in the code. You can experiment with them or create your own versions.

We can also create simulations of natural phenomena, like water or smoke using the base particle system we explored in this activity. For instance, imagine a smoke simulation emerging from an incense stick. By adjusting parameters such as colour, velocity, shape, size, and lifespan of the particles, we can achieve different effects. Using the same code, try experimenting with these parameters to create your smoke simulation.



In this lesson, we explored simulations and their applications, focusing on enhancing a particle system to realistically simulate flames using the noise() function based on Perlin Noise. We experimented with generating different types of flames by adjusting input parameters, demonstrating how Perlin Noise can create natural and fluid visual effects.

Exercise

State if the statements are True or False.

1. The Alpha (A) value in the RGBA colour system represents the opacity or transparency of the colour.

   _____

2. The length of the array x can be found using x.length.

   _____

3. The index of the last element of an array x will be x.length.

   _____

Tick mark the correct answer.

4. The _____ command creates random numbers generated using the Perlin noise.

   a. noise() b. millis() c. random() d. map()

5. A command that maps a value in a variable from one range to another is _____.

   a. noise() b. map() c. random() d. None of the above

6. We can add objects created from a class into an array using the _____ command.

   a. add() b. mult() c. push() d. Both b and c

Exercise

Answer the following questions in one line.

7 What is the significance of the Perlin noise?

8 What are simulations?

Tech Flash

Simulation	:	It is the process of creating a model or representation of a real-world system or phenomenon.
Perlin Noise	:	It is a type of smooth, continuous noise used in computer graphics to generate natural-looking patterns.
noise()	:	The noise() command generates random values based on the Perlin noise.
map()	:	The map() command maps a number from one range to another.